ANJALAI AMMAL MAHALINGAM ENGINEERING COLLEGE

Tamil Nadu Skill Development Corporation Project Documentation

Multi-Line Insurance Policy and Claims Management System

Team Details

S.No	Name	Reg.No	Mail ID
1	ABINASH M	820423205001	abhiandabi0707@gmail.com
2	ABISHEK S	820423205002	abishak51025@gmail.com
3	AJAY V	820423205007	ajaykandiyar007@gmail.com
4	BALAJI S L	820423205017	slbalaji747@gmail.com
5	KISHORE R	820423205043	kishorerajendran777@gmail.com

Multi-Line Insurance Policy and Claims Management System

User Story:

Multi-Line Insurance Policy and Claims Management System is designed to provide a centralized Salesforce-based solution for managing Vehicle, Property, and Life Insurance policies and claims. The system enables insurance agents to create and manage policy information using a flexible data model with policy-specific Record Types and Field Sets. Guided Screen Flows simplify policy quoting, while Apex automation calculates premiums based on policy and vehicle details. Claims are automatically routed to the appropriate queue based on policy type, and high-value claims are processed through an approval workflow involving Senior Adjusters and Claims Managers. An LWC-based Claims Dashboard provides adjusters with a focused view of their assigned claims, including claim amount, policy type, policy holder, status, and days open. This initiative improves policy processing efficiency, claim handling, data accuracy, operational visibility, and customer service.

1. INTRODUCTION

1.1 Project Overview

The Multi-Line Insurance Policy and Claims Management System is a Salesforce-based solution designed to centralize and streamline the management of Vehicle, Property, and Life Insurance policies and claims. The system replaces fragmented and manual processes with structured Salesforce objects, Record Types, Field Sets, Flows, Apex automation, Approval Processes, and Lightning Web Components.

The solution enables insurance agents to create and manage policies through a flexible data model, while guided Screen Flows simplify the policy quoting process. The system validates policy information and uses Apex automation to calculate premiums based on factors such as state and model year. Once a claim is created, automation identifies the related policy type and routes the claim to the appropriate claims queue.

For claims adjusters, an LWC-based Claims Dashboard provides visibility into assigned claims, including claim number, policy type, policy holder, claim amount, approval status, and days open. High-value claims exceeding the defined threshold are processed through a structured approval workflow involving Senior Adjusters and Claims Managers. Security and access are managed using sharing rules and role-specific Permission Sets.

Overall, the project provides a centralized platform for improving policy processing, premium calculation, claim routing, claim approval, data accuracy, operational visibility, and customer service across multiple insurance lines.

1.2 Purpose

The purpose of the Multi-Line Insurance Policy and Claims Management System is to provide a centralized Salesforce-based platform for managing Vehicle, Property, and Life Insurance policies and claims. The system is designed to overcome the limitations of legacy processes that result in manual, slow, and inconsistent quoting, policy issuance, and claims handling.

The solution aims to standardize and accelerate policy quoting and issuance, automate claim initiation and routing based on policy type and claim severity, and provide a 360-degree view of Customer, Policy, and Claim information. It also supports guided quoting through Screen Flows, automated premium and claim processing, data validation, and an LWC-based dashboard to improve claims adjuster efficiency.

Overall, the project focuses on improving operational efficiency, data accuracy, claims processing, and customer experience by bringing multiple insurance lines into a unified Salesforce solution.

2. IDEATION PHASE

2.1 Problem Statement (Customer Problem Statement Template)

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel

PS-1	Insurance Operations Manager	standardize policy quoting and issuance	the process is manual, slow, and inconsistent	legacy systems and processes are fragmented	frustrated and concerned
PS-2	Insurance Agent	create and manage policies efficiently	different insurance lines require different information	Vehicle, Property, and Life policies have diverse data requirements	overwhelmed
PS-3	Insurance Agent	provide accurate policy quotes	quoting requires multiple steps and manual processing	there is no unified quoting platform	frustrated and uncertain
PS-4	Claims Manager	manage claims efficiently	claim review and routing are largely manual	claims processes are not centralized or automated	stressed
PS-5	Claims Adjuster	manage assigned claims effectively	I lack a focused view of my claims workload	claim information is not easily available in one place	frustrated and less productive
PS	**I am (Customer)**	**I'm trying to**	**But**	**Because**	**Which makes me feel**
PS-6	Claims Manager	route claims to the appropriate team	claims need to be handled according to policy type and claim severity	routing processes are complex and require automation	concerned about delays
PS-7	Senior Adjuster	process highvalue claims efficiently	approval requires multiple levels of review	high-value claims require structured approval and escalation	pressured
PS-8	Insurance Operations Manager	maintain accurate policy information	different products require different fields and validations	Vehicle, Property, and Life policies have varying data requirements	concerned about data quality
PS-9	Customer	get my policy and claim handled quickly	manual processing causes delays	policy and claims processes are fragmented	uncertain and dissatisfied

PS-10	Insurance Management	get a complete view of customer, policy, and claim information	information is spread across processes and systems	there is no unified platform	concerned about operational efficiency

2.2 Empathy Map Canvas

User Persona: Insurance Claims Adjuster

- **Think & Feel** – Concerned about claim delays and workload
- **Hear** – Customer and team concerns about claim resolution
- **See** – Multiple claims, policy details, and approval statuses
- **Say & Do** – Reviews claims and follows up on approvals
- **Pain** – Manual processing, complex routing and limited visibility
- **Gain** – Claims dashboard, automation and faster approvals

2.3 Brainstorming & Idea Prioritization

Ideas Considered

- **Manual Registers** – Policy and claim information maintained manually
- **Spreadsheet Tracking** – Policies, claims, and approvals tracked using spreadsheets
- **Standalone Portal** – Separate portal for policy and claim management
- **Salesforce CRM** – Unified platform for policies, claims, automation, and reporting

Final Selection

Salesforce CRM was selected due to:

- **Automation** – Automates quoting, premium calculation, claim routing, and approval processes
- **Scalability** – Supports multiple insurance lines including Vehicle, Property, and Life Insurance
- **Real-Time Visibility** – Provides centralized visibility into customer, policy, and claim information
- **Unified Platform** – Brings policy and claims processes together in one system
- **Improved Efficiency** – Reduces manual processing and supports faster claim handling

3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

Stages:

- Insurance Inquiry
- Policy Selection – Vehicle / Property / Life
- Quote Request
- Policy Information Capture
- Premium Calculation
- Policy Issuance
- Claim Initiation
- Claim Routing
- Claim Review & Approval
- Claim Resolution
- Customer Confirmation
- Reporting & Visibility

3.2 Solution Requirements

Functional Requirements

FR No	Functional Requirement
FR-01	The system shall allow users to create and manage Insurance Policy records for Vehicle, Property, and Life Insurance.
FR-02	The system shall support separate Record Types for Vehicle, Property, and Life policies.
FR No	**Functional Requirement**
FR-03	The system shall capture product-specific policy information such as VIN for Vehicle Insurance and Square Footage/Year Built for Property Insurance.
FR-04	The system shall provide a guided Auto Quoting Screen Flow to capture policy holder, policy start date, state, VIN, and model year.
FR-05	The system shall create a draft policy during the quoting process.
FR-06	The system shall calculate policy premiums using Apex-based premium calculation logic.
FR-07	The system shall validate Vehicle Insurance VINs and ensure that the VIN contains exactly 17 characters.

FR-08	The system shall allow users to create and manage Claim records associated with policies.
FR-09	The system shall support different Claim Record Types for Accident, Property, and Life claims.
FR-10	The system shall automatically route claims to the appropriate queue based on the related policy type.
FR-11	The system shall identify high-value claims based on the configured claim amount threshold.
FR-12	The system shall submit qualifying high-value claims through a multi-level approval process involving Senior Adjusters and Claims Managers.
FR-13	The system shall provide an Approve/Reject Claim capability with approver comments.
FR-14	The system shall provide Claims Adjusters with an LWC Claims Dashboard showing assigned claims.
FR-15	The dashboard shall display information such as Claim Number, Policy Type, Policy Holder, Claim Amount, Status, and Days Open.
FR-16	The dashboard shall allow users to filter claims by All, Auto, Property, and Life.
FR-17	The system shall provide automated updates to related Policy and Claim records through Record-Triggered Flows.
FR-18	The system shall support state-based access/sharing for applicable insurance users and records.
FR-19	The system shall provide role-specific access through Permission Sets for Insurance Agents, Claims Adjusters, and Claims Managers.
FR-20	The system shall support reporting and visibility of Customer, Policy, and Claim information from the centralized Salesforce platform.

Non-Functional Requirements

The following requirements describe the expected quality, usability, security, scalability, and maintainability of the solution.

ID	Non-Functional Requirement	Description
NFR-01	Usability	The system should provide simple, guided Screen Flows that minimize unnecessary clicks and make policy quoting easy for agents.

NFR-02	Performance	Policy quoting, premium calculation, claim routing, and dashboard operations should execute efficiently to support day-to-day insurance operations.
NFR-03	Scalability	The solution should support the three insurance lines—Vehicle, Property, and Life—and allow additional insurance products to be accommodated in the future.
NFR-04	Data Integrity	Policy and claim information should be validated to maintain accurate and consistent data across insurance products.
NFR-05	Security	Access to insurance information should be controlled according to the user's responsibility, including state-based sharing and role-specific Permission Sets.
NFR-06	Maintainability	The solution should use modular Salesforce components such as Flows, Apex classes, LWC components, Record Types, and Field Sets so that individual components can be maintained independently.
NFR-07	Reliability	Automated claim routing, premium calculation, and approval processes should consistently execute according to the configured business rules.
NFR-08	Extensibility	The data model should accommodate different productspecific fields without requiring completely separate structures for each insurance line.
NFR-09	Visibility	Users should have timely access to relevant Customer, Policy, and Claim information through centralized Salesforce records and the Claims Dashboard.
NFR-10	User Adoption	The application should provide an intuitive experience that reduces manual effort and supports efficient adoption by agents and claims users.

3.3 Data Flow Diagram

Flow:

Customer / Agent → Screen Flow → Policy Validation → Policy__c → PremiumCalculator → Claim__c → Claim Routing Flow → Claims Queue → Claims Adjuster → Approval Process → Claim Update → LWC Dashboard / Reports

Main Data Flow Step 1 – Customer / Agent Input

→ Customer and policy information is provided by the Insurance Agent.

Step 2 – Policy Data Capture

→ AutoQuoting Screen Flow captures policy holder, policy start date, state, VIN, and model year.

Step 3 – Policy Validation

→ Salesforce validates the entered information, including the 17-character VIN requirement for Auto policies.

Step 4 – Policy Creation

→ Valid information is stored in the Policy__c object with the appropriate Vehicle, Property, or Life Record Type.

Step 5 – Premium Calculation

→ Policy information is passed to PremiumCalculator Apex to calculate the applicable premium.

Step 6 – Claim Creation

→ When a claim occurs, the user creates a Claim__c record linked to the relevant Policy.

Step 7 – Claim Routing

→ A Record-Triggered Flow identifies the policy type and routes the claim to the appropriate Auto, Property, or Life Claims Queue.

Step 8 – Claim Review

→ The Claims Adjuster accesses assigned claims through the Claims Dashboard and reviews policy and claim information.

Step 9 – High-Value Claim Approval

→ Claims exceeding the configured threshold are submitted for approval to the Senior Adjuster → Claims Manager.

Step 10 – Claim Decision & Update

→ The approver selects Approve/Reject and provides comments; the claim status is updated.

Step 11 – Dashboard & Reporting

→ Claim information is displayed through the LWC Claims Dashboard, providing visibility into claim amount, policy type, policy holder, status, and days open.

3.4 Technology Stack

Layer	Technology
UI	Salesforce Lightning, Lightning Web Components (LWC)
Logic	Salesforce Flows, Apex
Database	Salesforce Custom Objects (Policy__c, Claim__c), Contact, User
Layer	**Technology**
Automation	Record-Triggered Flows, Validation Rules, Approval Processes
Processing	Apex Premium Calculator, Apex Controllers
Data Configuration	Record Types, Field Sets, Custom Fields
Security	Permission Sets, State-Based Sharing Rules

Reporting	Salesforce Reports & Dashboards
Testing	Apex Test Classes

4. PROJECT DESIGN PHASE

4.1 Problem–Solution Fit

Salesforce is a strong fit for the Multi-Line Insurance Policy and Claims Management System because it directly addresses the project's key challenges through configurable automation, flexible data management, secure access, and real-time visibility. The solution combines Screen Flows, Record Types, Field Sets, Apex, Record-Triggered Flows, Approval Processes, and LWC to support policy quoting, premium calculation, claim routing, claim approval, and adjuster operations.

4.2 Proposed Solution

Parameter	Description
Problem	Manual and inconsistent policy quoting, issuance, and claims handling
Solution	Salesforce CRM-based Insurance Policy & Claims Management System
Innovation	Automated quoting, premium calculation, claim routing, and highvalue claim approval
Impact	Faster policy processing, efficient claim handling, and improved customer service
Business Model	Centralized management of Vehicle, Property, and Life Insurance
Scalability	Flexible data model supporting multiple insurance products and future expansion
Visibility	Centralized Customer, Policy, and Claim information with Claims Dashboard
Security	Role-based and state-based access using Permission Sets and Sharing Rules

The proposed solution is aligned with the project's documented objective of creating a single Salesforce platform to standardize policy processes, automate claims, and provide a 360-degree view of Customer, Policy, and Claim data.

4.3 Solution Architecture

The Multi-Line Insurance Policy and Claims Management System uses a layered Salesforce architecture to manage policy quoting, premium calculation, claims processing, approvals, security, and reporting.

Key Architecture Components

Layer	Project Implementation
User Layer	Insurance Agents, Claims Adjusters, Claims Managers
UI Layer	Salesforce Lightning, Screen Flows, LWC
Automation Layer	Record-Triggered Flows, Approval Processes, Validation Rules
Business Logic	Apex Premium Calculator and Claims Controller
Data Layer	Policy and Claim custom objects with Vehicle, Property, and Life Record Types
Security Layer	Permission Sets and state-based Sharing Rules
Reporting Layer	Claims Dashboard, Reports and Salesforce Dashboards

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

- Agile methodology
- Epics → Stories → Story Points
- Sprint-based execution
- Velocity calculation used for estimation

Product Backlog, Sprint Schedule, and Estimation

Sprint	Functional Requirement (Epic)	User Story No.	User Story / Task	Story Points	Priority	Team Member
Sprint1	Data Modeling	US-01	As an admin, I want to create Policy and Claim objects so that insurance policy and claim information can be stored and managed systematically.	5	High	Mohamed Anwar S

Sprint	Functional Requirement (Epic)	User Story No.	User Story / Task	Story Points	Priority	Team Member
Sprint1	Data Modeling	US-02	As an admin, I want to configure Vehicle, Property, and Life Record Types so that different insurance products can be managed using a unified Policy structure.	3	High	Ramya M
Sprint1	Data Modeling	US-03	As an admin, I want to configure policy Field Sets and product-specific fields so that diverse insurance information can be captured efficiently.	5	High	Amirthavarshini A
Sprint1	Quoting & Automation	US-04	As an insurance agent, I want a guided Auto Quoting Screen Flow so that I can capture policyspecific information through a standardized quoting process.	8	High	Dharshini S
Sprint2	Validation	US-05	As an insurance agent, I want VIN validation during vehicle quoting so that incorrect vehicle information can be prevented.	3	High	Dhayal D
Sprint2	Apex	US-06	As an insurance system, I want PremiumCalculator Apex to calculate premiums so that premium values are generated accurately.	8	High	Mohamed Anwar S
Sprint2	Automation	US-07	As an insurance agent, I want the premium calculation integrated with the quoting Flow so that the calculated premium is available during the quote process.	5	High	Ramya M

Sprint2	Claims Automation	US-08	As a claims manager, I want claims to be routed based on policy type so that claims are assigned to the appropriate support queue.	8	High	Amirthavarshini A
Sprint	**Functional Requirement (Epic)**	**User Story No.**	**User Story / Task**	**Story Points**	**Priority**	**Team Member**
Sprint3	Apex	US-09	As a claims adjuster, I want a ClaimsAdjusterController to retrieve my assigned claims so that I can efficiently manage my claim workload.	8	High	Dharshini S
Sprint3	LWC Dashboard	US-10	As a claims adjuster, I want a claims dashboard and reusable claim tile so that I can view and manage assigned claims efficiently.	8	High	Dhayal D
Sprint4	Approval	US-11	As a claims manager, I want high-value claims to go through an approval process and approver Flow so that high-value claims receive appropriate review.	8	High	Mohamed Anwar S
Sprint4	Security & Testing	US-12	As a system administrator, I want to implement tests, sharing rules, and Permission Sets so that the solution is validated and users receive appropriate access.	8	High	Ramya M

Project Tracker, Velocity & Burndown Chart

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed	Sprint Release Date (Actual)
Sprint-1	21	6 Days	[dd Mon yyyy]	[dd Mon yyyy]	21	[dd Mon yyyy]
Sprint-2	24	6 Days	[dd Mon yyyy]	[dd Mon yyyy]	24	[dd Mon yyyy]

Sprint-3	16	6 Days	[dd Mon yyyy]	[dd Mon yyyy]	16	[dd Mon yyyy]
Sprint-4	16	6 Days	[dd Mon yyyy]	[dd Mon yyyy]	16	[dd Mon yyyy]

Average velocity = total story points / number of sprints = 77 / 4 ≈ 19.25 story points per sprint.

6. PROJECT DEVELOPMENT PHASE

What you will learn

- Data Modelling
- Fields and Relationships
- Lightning App Builder
- Record Triggered Flows
- Apex and Apex Triggers
- Batch Apex
- Scheduled Apex

Milestone 1: Core Data Model & Policy Configuration

User Story 1: Creation of custom objects 1. Create Policy custom object

- Go to Setup, click on Object Manager.
- Click on Create and select Custom Object.
- Fill the following details:
  - Label: Policy
  - Plural Label: Policies
  - Object Name: Auto Populated
  - Record Name: Policy Number
  - Data Type: Auto Number
  - Format: P-{0000}
  - Starting Number: 1
- Under Optional Features: Allow Reports, Activities and Track Field History.
- Under Deployment Status: select the Deployed radio button.
- Under Search Status: check the Allow Search checkbox.
- Click on Save.

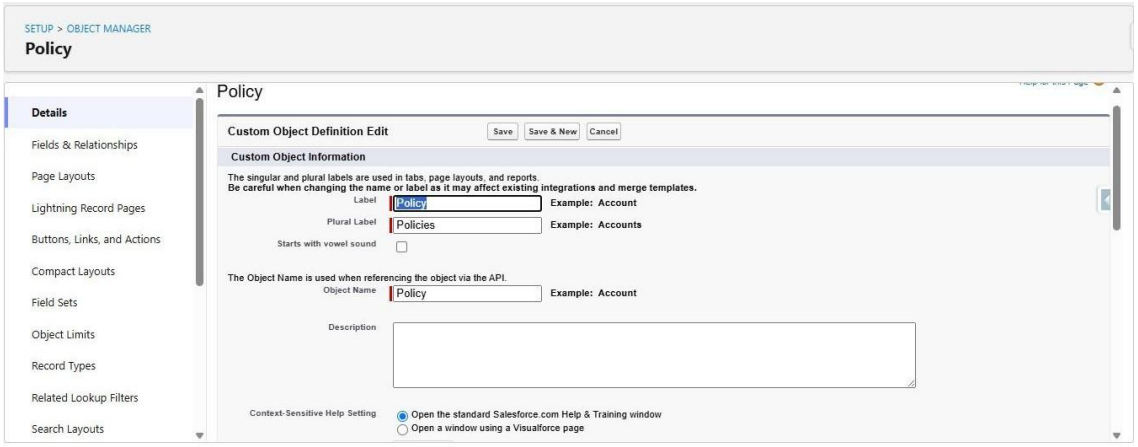


Figure 1: Creating the Policy custom object

2. Define Record Types on Policy object

- Go to Setup, click on Object Manager.
- Search for the Policy object and click on Policy__c.
- In the left side panel, click on Record Types and click on New.
- Fill out the following details:
  - Select the existing record type as Master.
  - Give the record type label as “Auto”.
  - Record Type Name will be auto populated.
  - Give a meaningful description.
  - Check the Active checkbox.
  - Check the Make Available checkbox for all the profiles.
  - Click on Next.
- On the Assign Page Layouts page, leave everything as is and click on Save.
- Repeat the same steps for creating the Property and Life record types.

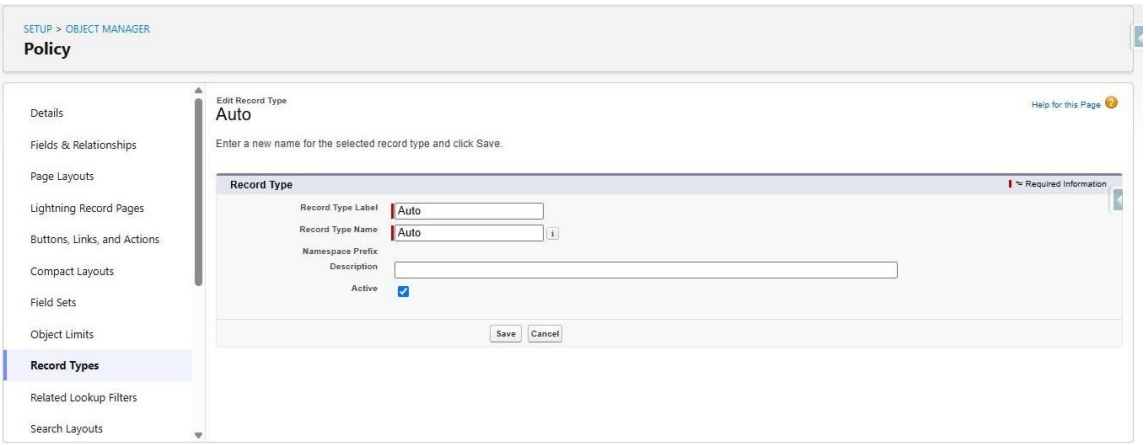


Figure 2: Auto record type on the Policy object

3. Create Field Sets for policy-specific data

- Go to Setup, click on Object Manager.
- Search for the Policy object and click on Policy__c.
- In the left side panel, click on Field Sets and click on New.
- Fill out the following details:
  - Field Set Label as Vehicle.
  - Field Set Name will be auto populated.
  - Where is this used: In the Policy object.
  - Click on Save.
- Click on the field set you have just created; it will redirect to the page layout setup.
- Drag the VIN and Model Year fields to the field set and click on Save.

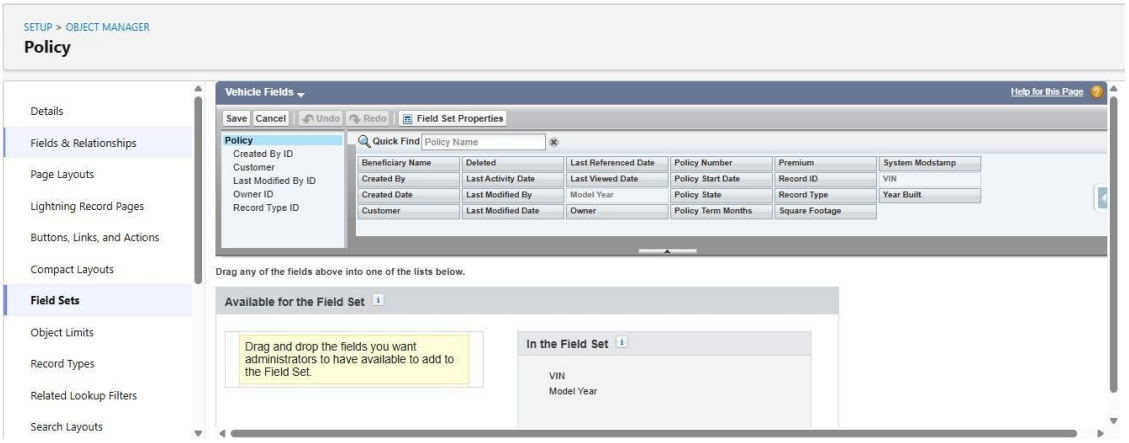


Figure 3: Vehicle field set (VIN, Model Year) • Repeat the same steps for creating the Property and Life field sets.

3.1 Property Field Set requires:

- Square Footage
- Year Built

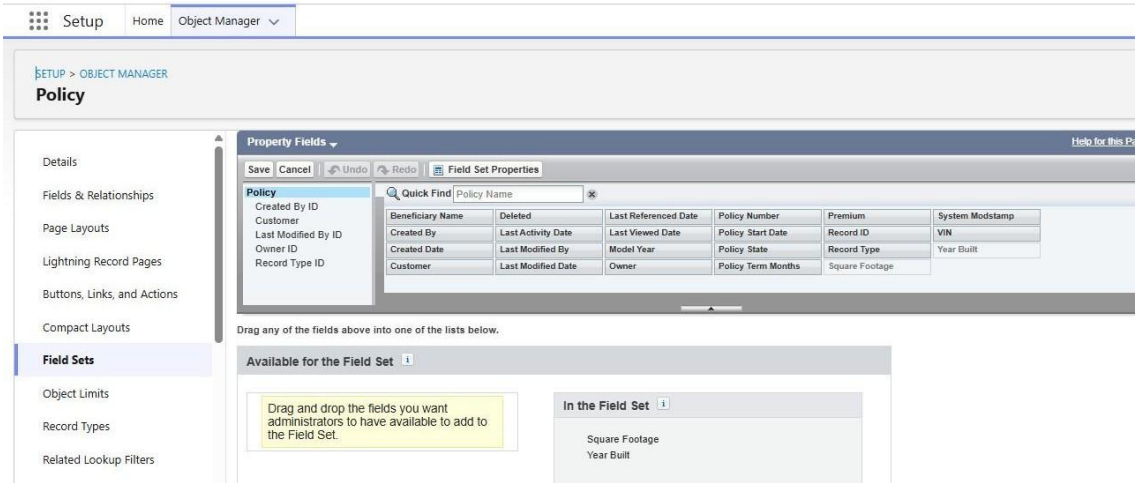


Figure 4: Property field set (Square Footage, Year Built)

3.2 Life Field Set requires:

- Beneficiary Name
- Policy Term Months

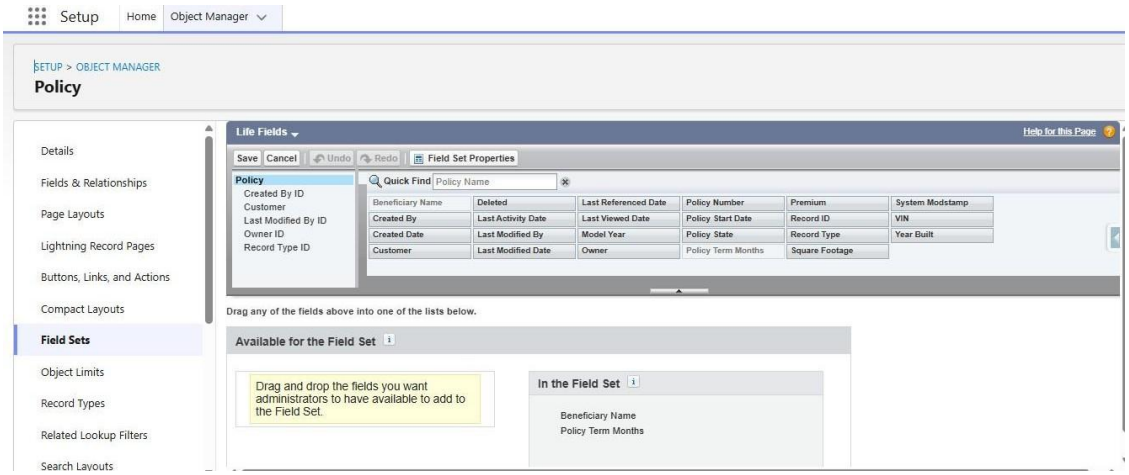


Figure 5: Life field set (Beneficiary Name, Policy Term Months)

4. Create Fields in the Policy object

S.No	Name of the Field	Data Type	Required
1	Beneficiary Name	Text(50)	
2	Customer	Lookup(Contact)	
3	Model Year	Text(5)	
4	Policy Start Date	Date	
5	Policy Term Months	Number(18, 0)	
6	Premium	Currency(16, 2)	
7	Square Footage	Number(18, 0)	
8	VIN	Text(20)	
9	Year Built	Text(5)	

5. Create Claim custom object

- Go to Setup, click on Object Manager.
- Click on Create and select Custom Object.
- Fill the following details:
  - Label: Claim
  - Plural Label: Claim
  - Object Name: Auto Populated
  - Record Name: Claim Number

- Data Type: Auto Number
- Format: C-{0000}
- Starting Number: 1

- Under Optional Features: Allow Reports, Activities and Track Field History.
- Under Deployment Status: select the Deployed radio button.
- Under Search Status: check the Allow Search checkbox.
- Click on Save.

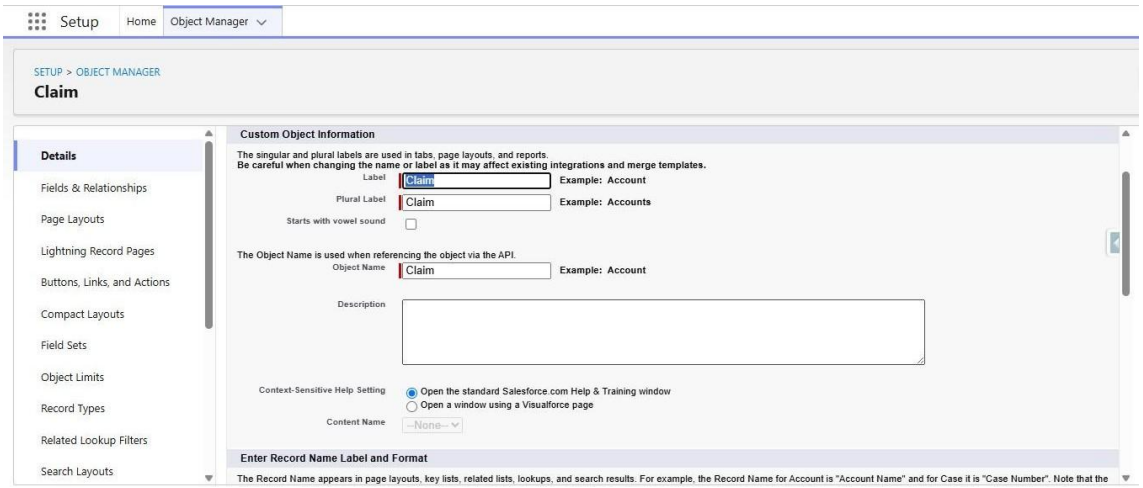


Figure 6: Creating the Claim custom object

6. Define Record Types on Claim object

- Go to Setup, click on Object Manager.
- Search for the Claim object and click on Claim__c.
- In the left side panel, click on Record Types and click on New.
- Fill out the following details:
  - Select the existing record type as Master.
  - Give the record type label as “Accident”.
  - Record Type Name will be auto populated.
  - Give a meaningful description.
  - Check the Active checkbox.
  - Check the Make Available checkbox for all the profiles.
  - Click on Next.
- On the Assign Page Layouts page, leave everything as is and click on Save.
- Repeat the same steps for creating the Property and Life record types.

7. Create Fields in the Claim object

S.No	Field Name	Data Type	Required

1	Adjuster	Lookup(User)	
S.No	**Field Name**	**Data Type**	**Required**
2	Approval Status	Picklist(New, Submitted for Approval, Approved, Rejected)	
3	Claim Amount	Currency(16, 2)	
4	Date of Loss	Date/Time	
5	Description	Long Text Area	
6	Policy	Lookup(Policy)	

8. Create an AutoQuoting Screen Flow

- Go to Setup, search for Flows in the Quick Find box, click on Flows and click on New Flow.
- Select Screen Flow and click on Create.
- Add a Get Records element immediately after the Start element:
  - Label: Get Vehicle RT ID (API Name auto populated).
  - Select Record Type for Object.
  - For Conditional Requirements, select All Conditions Are Met.
  - Conditions: sObject Type Name Equals Policy__c AND Record Type Name Equals Auto.

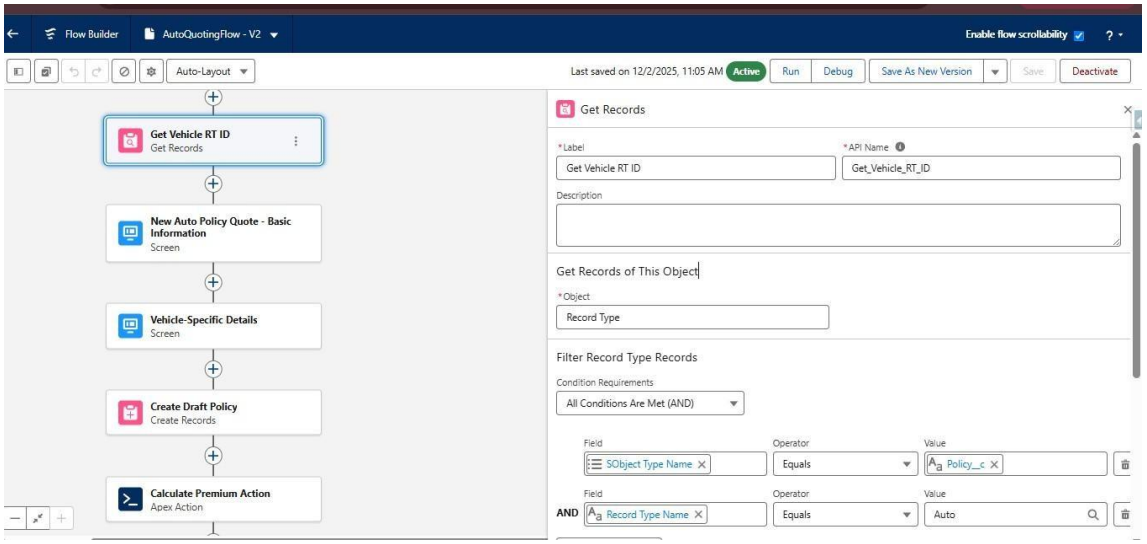


Figure 7: AutoQuotingFlow - Get Vehicle RT ID element • Add a Screen element immediately after the Get Records element:

- Label: New Auto Policy Quote - Basic Information.
- Drag the Display Text component and give the API Name as Screen_1_Policy_Basics.
- Drag the Lookup component: API Name Customer Lookup, Field API Name Customer__c, Label Policy Holder, ObjectApiName Policy__c.

    - Drag the Date component: Label Policy Start Date.
    - Drag the Picklist component: Label Policy State; for Choice, create a new resource of type Picklist Choice Set, select Policy as the object, Picklist as the data type and Policy State as the field, then Save.
    - Click on Done.
- Add another Screen element after the Basic Information screen:
    - Label: Vehicle-Specific Details.
    - Drag the Display Text component and give the API Name as Screen_2_Vehicle_Details.
    - Drag a Text component with the label VIN.
    - Drag another Text component with the label Model Year.
    - Click on Done.
- Add a Create Records element after the screen element:
    - Label: Create Draft Policy; store field values manually; select the Policy object.
    - Customer → Basic Information screen → Policy Holder → Record ID
    - Model Year → Vehicle Information screen → Model Year
    - Policy Start Date → Basic Information screen → Policy Start Date
    - Policy State → Basic Information screen → Policy State
    - RecordType ID → Get Vehicle RT ID → RecordType ID
    - VIN → Vehicle Information screen → VIN
    - Check the manually assign variables checkbox and create a new variable resource with API Name varPolicyId and Data Type Text, then click Done.
- Save the flow as AutoQuotingFlow and activate the flow.

9. Create Validation Rule on Policy object

- Go to Object Manager, search for the Policy object and click on it.
- In the left side panel, scroll down, click on Validation Rules and click on New.
- Give the rule name as VIN_Must_Be_17_Characters and check the Active checkbox.
- Write the following formula in the formula editor:
    - AND(RecordType.DeveloperName = "Auto", LEN(VIN__c) <> 17)
- Error message: The Vehicle Identification Number (VIN) must be exactly 17 characters long for Auto Policies.
- Select the error location as Field and select the VIN field.
- Click on Save.

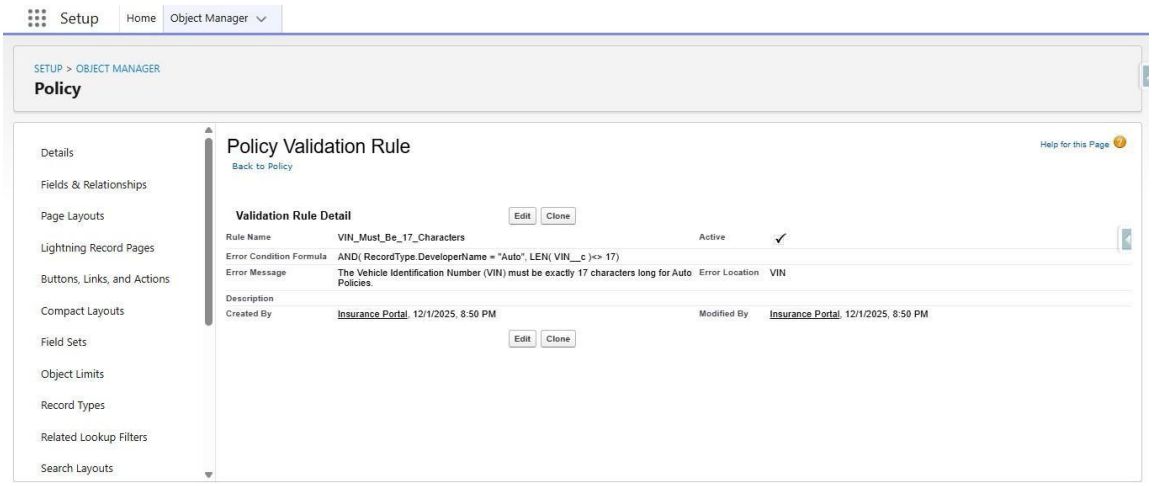


Figure 8: VIN_Must_Be_17_Characters validation rule

Milestone 2: Complex Policy Issuance & Claim Routing Automation

1. Create an Apex Class (PremiumCalculator) with a method that accepts policy data

- Click on the Setup icon, right-click on Developer Console and open the link in a new tab.
- At the top left corner, click on File → New → Apex Class, give the name PremiumCalculator and click on Create.
- Paste the following code in the Apex class:

PremiumCalculator.cls

```
public class PremiumCalculator {
    public class PremiumInputWrapper {
        @InvocableVariable(required=true) public Id
        policyId;
    }
    @InvocableMethod(label='Calculate Policy Premium' description='Calculates premium based on
    policy fields (VIN, Model Year, State).')
    public static List<Decimal> calculatePremium(List<PremiumInputWrapper> inputs) {
        List<Decimal> premiumResults = new List<Decimal>();
        Set<Id> policyIds = new Set<Id>(); for
        (PremiumInputWrapper input : inputs) {
            if (input.policyId != null) {
                policyIds.add(input.policyId);

            }
        }
        List<Policy__c> policiesToRate = [
            SELECT Id, VIN__c, Model_Year__c, Policy_State__c FROM
            Policy__c
            WHERE Id IN :policyIds
        ];
        for (Policy__c policy : policiesToRate) {
            Decimal basePremium = 1000.00; if
            (policy.Policy_State__c == 'CA') {
                basePremium *= 1.15;
```

```
        } else if (policy.Policy_State__c == 'TX') {
            basePremium *= 1.05;

        } if (policy.Model_Year__c != null && Integer.valueOf(policy.Model_Year__c) < 2018)
        {
            basePremium += 100;

        }
        premiumResults.add(basePremium);
    }
    return premiumResults;
  }

}
```

- Press Ctrl + S to save the Apex class.
- This Apex class needs to be invoked in the AutoQuotingFlow.

2. Integrate the Apex action into the final step of the AutoQuotingFlow

- Go to Setup, search for Flows in the Quick Find box, find AutoQuotingFlow and open it in Flow Builder.
- After the Create Records element, add an Action element:
  - Label: Calculate Premium Action (API Name auto populated).
  - Under Set Input Values, set Policy Id to the varPolicyId variable created in the previous step.
  - Under View Output Resources, check manually assign variables and create a new variable resource: API Name varCalculatedPremium, Data Type Currency, Decimal Places 2, then click Done.

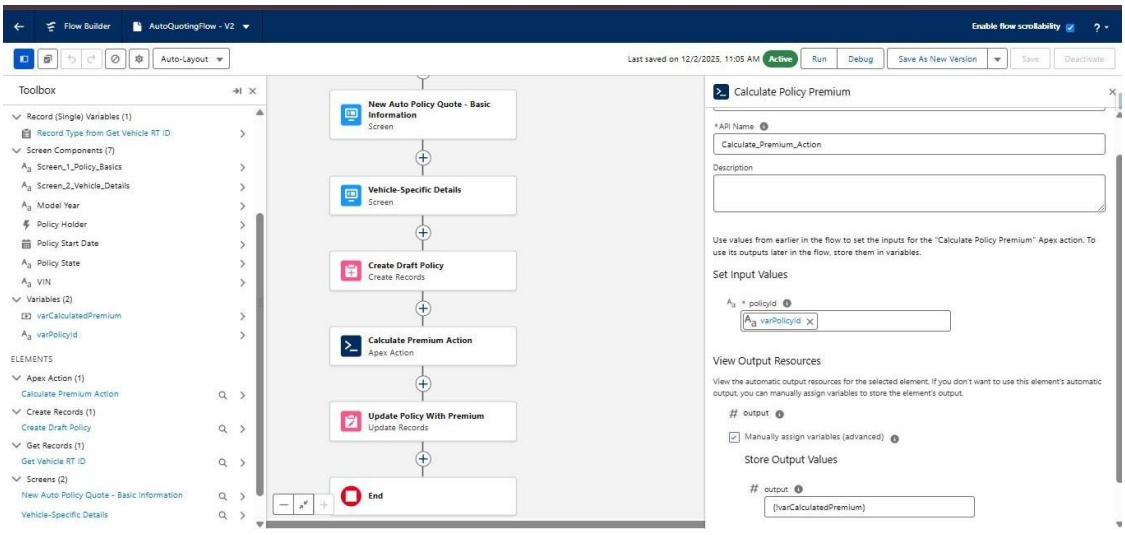


Figure 9: Calculate Premium Action in AutoQuotingFlow

- After the Action element, add an Update Records element:
  - Label: Update Policy With Premium.

◦ Select Specify conditions to identify records, and set fields individually.

◦ Select the Policy object and All Conditions Are Met.

◦ Condition: Record Id Equals varPolicyId variable.

◦ Under Set Field Values for the Policy record, assign the varCalculatedPremium variable to the Premium field.

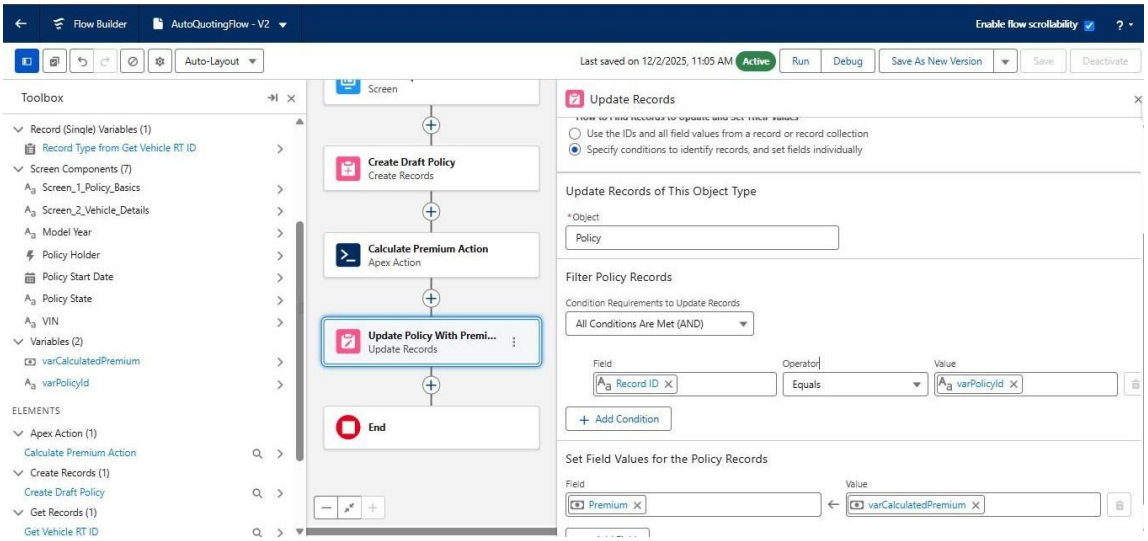


Figure 10: Update Policy With Premium element

- Save the flow as a new version and activate the flow.

3. Create a Record-Triggered Flow (After Save) on the Claim object

- Go to Setup, search for Flows in the Quick Find box, click on New Flow, select RecordTriggered Flow and click on Create.
- Select Claim as the object and configure the trigger as: A record is created.
- Optimize the flow for Actions and Related Records.

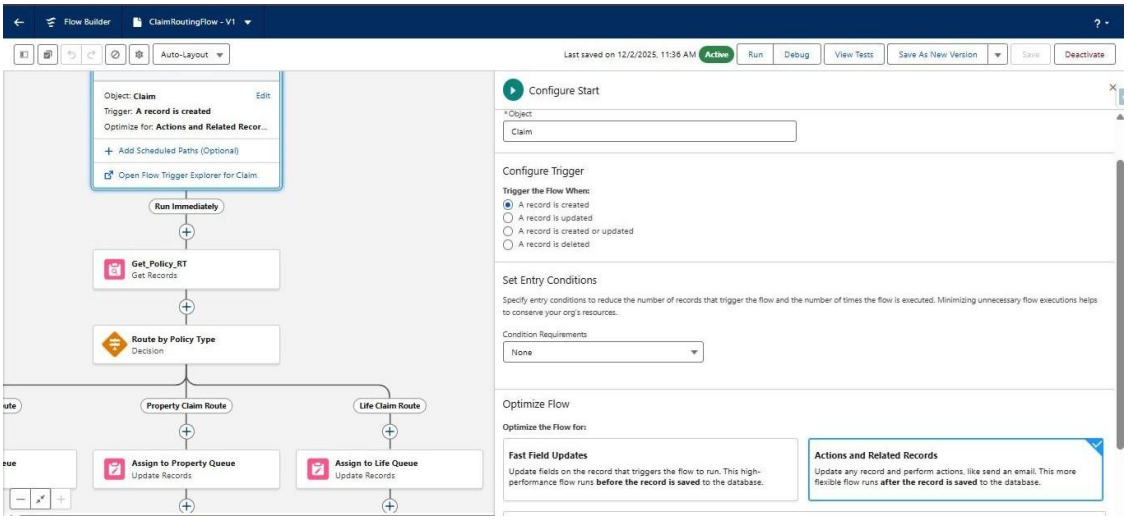


Figure 11: Claim routing flow - trigger configuration

- Add a Get Records element after the Start element:
  - ◦ Label: Get_Policy_RT; select the Policy object; All Conditions Are Met.

◦ Condition: Record Id Equals Triggering Claim → Policy.

◦ Store only the first record; choose fields and assign variables (separate variables).

◦ For the recordTypeId field, create a variable resource with API Name varPolicyRTName and Data Type Text, then click Done.

- Add a Decision element after the Get Records element:
  - ◦ Label: Route by Policy Type; Decision Logic: Manually set conditions.
  - ◦ Outcome 1: Label Auto Claim Route; All Conditions Are Met; varPolicyRTName equals Auto.
  - ◦ Outcome 2: Label Property Claim Route; All Conditions Are Met; varPolicyRTName equals Property.
  - ◦ Default Outcome: Label Life Claim Route.

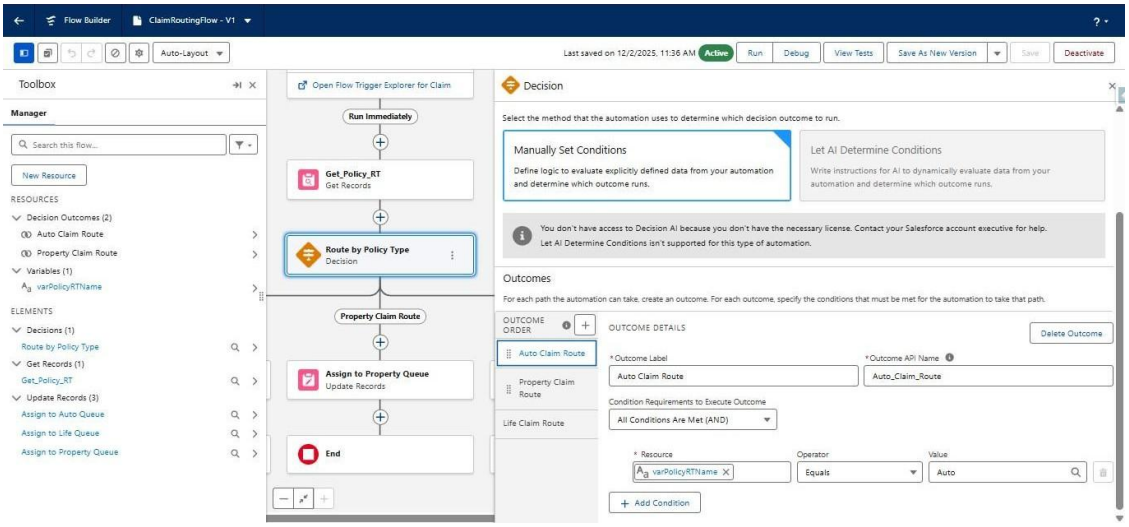


Figure 12: Route by Policy Type decision outcomes

- Under the Auto Claim Route path, add an Update Records element:
  - ◦ Label: Assign to Auto Queue; Specify conditions to identify records; object Claim; condition Record Id equals Triggering Claim → Record ID.
  - ◦ Set the hardcoded Auto queue 18-digit Id for the Owner ID field.

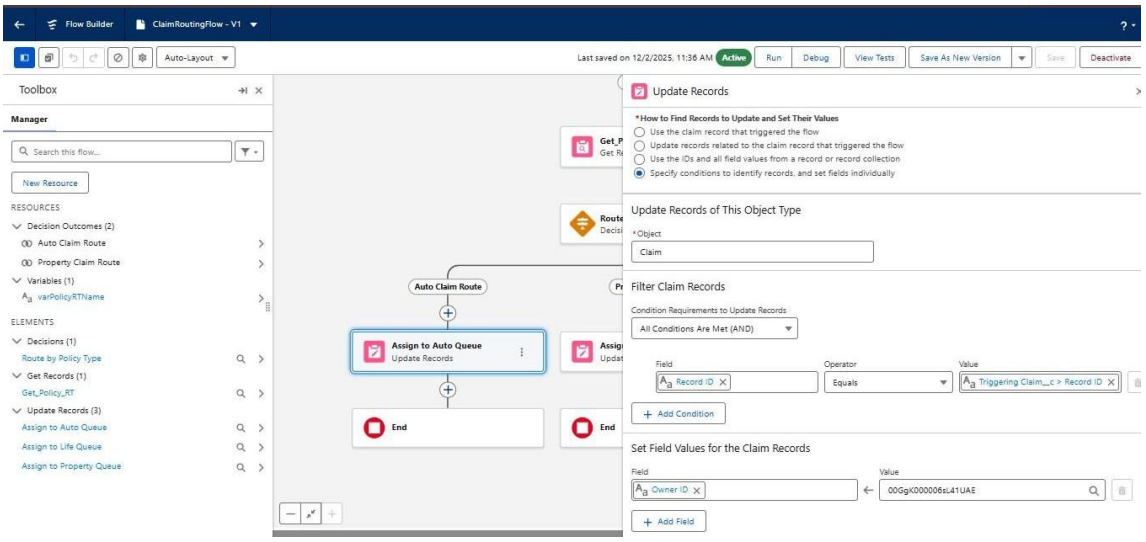


Figure 13: Assign to Auto Queue element

- Under the Property Claim Route path, add an Update Records element:
  - Label: Assign to Property Queue; same conditions as above.
  - Set the hardcoded Property queue 18-digit Id for the Owner ID field.
- Under the Life Claim Route path, add an Update Records element:
  - Label: Assign to Life Queue; same conditions as above.
  - Set the hardcoded Life queue 18-digit Id for the Owner ID field.

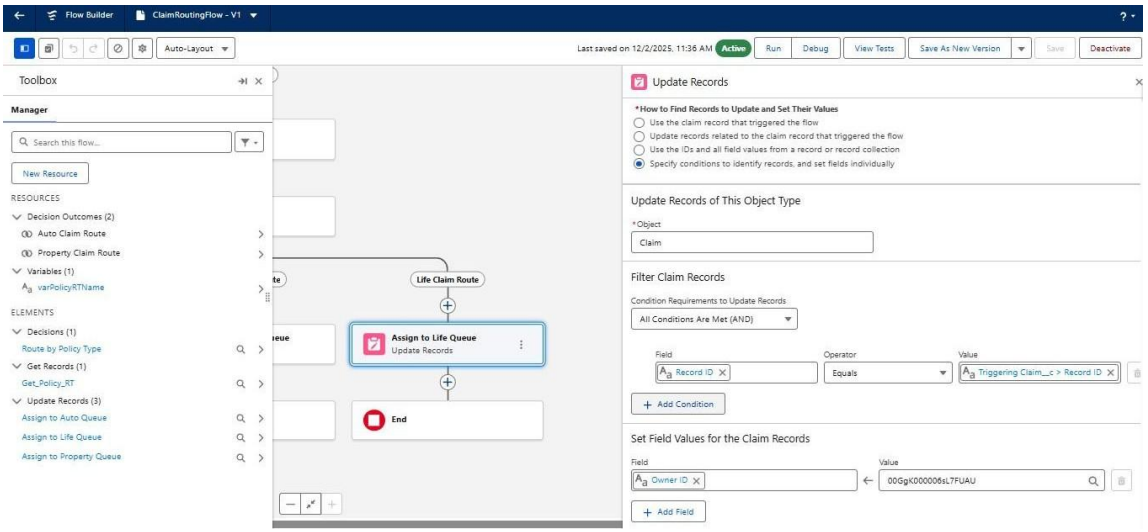


Figure 14: Assign to Life Queue element

Milestone 3: Claims Adjuster LWC Dashboard Development

1. Create an Apex class (ClaimsAdjusterController) returning a custom wrapper class

This class uses SOQL to query all Claims owned by the logged-in user, joining related Policy and Customer data, and returns a custom Wrapper Class listing key metrics.

- Click on the Setup icon, right-click on Developer Console and open the link in a new tab.

- Click on File → New → Apex Class, give the name ClaimsAdjusterController and click on Create.
- Paste the following code in the Apex class:

ClaimsAdjusterController.cls

```
public class ClaimsAdjusterController {
    public class ClaimWrapper {
        @AuraEnabled public Id claimId;
        @AuraEnabled public String claimNumber;
        @AuraEnabled public Decimal claimAmount;
        @AuraEnabled public String status;
        @AuraEnabled public String policyType;
        @AuraEnabled public String policyHolderName;
        @AuraEnabled public Integer daysOpen;

        public ClaimWrapper(Claim__c c, Policy__c p, Contact customerRec) {
            this.claimId = c.Id; this.claimNumber = c.Name;
            this.claimAmount = c.Claim_Amount__c; this.status =
            c.Approval_Status__c; Policy__c policyRecord = (Policy__c)p;
            if (policyRecord != null && policyRecord.RecordType != null) {


                this.policyType = policyRecord.RecordType.Name;

            } else {
                this.policyType = 'Unknown';

            }
            this.policyHolderName = (customerRec != null) ? customerRec.Name : ''; this.daysOpen =
            (c.Date_of_Loss__c != null)
                ? (Date.today().daysBetween(c.Date_of_Loss__c.date()))
                : 0;
        }
    }


    @AuraEnabled(cacheable=true)
    public static List<ClaimWrapper> getAssignedClaims() {
        List<ClaimWrapper> results = new List<ClaimWrapper>(); List<Claim__c> claims = [

            SELECT
                Id, Name, Claim_Amount__c, Approval_Status__c, Date_of_Loss__c, Policy__r.Name,
                Policy__r.RecordType.DeveloperName,
                Policy__r.Customer__r.FirstName,
                Policy__r.Customer__r.LastName

            FROM Claim__c
            WHERE OwnerId = :UserInfo.getUserId()
            ORDER BY Date_of_Loss__c DESC
        ];
        for (Claim__c claim : claims) {
```

```
        Policy__c policy = claim.Policy__r;

        Contact customer = (policy != null && policy.Customer__r != null)
            ? (Contact)policy.Customer__r
            : new Contact();
        results.add(new ClaimWrapper(claim, policy, customer));
    }
    return results;
  }

}
```

- Save the file using Ctrl + S.
- This Apex class is consumed by the LWC component.

2. Create the component claimsDashboardLwc (@wire service with client-side filtering)

- Open a new Chrome tab, search for the Lightning Studio extension, open the first link, click on Add to Chrome and confirm by clicking Add Extension.
- Open the Lightning Studio extension from your Salesforce org.
- At the top left corner, click on Create New and select Lightning Web Component.
- Enter the component name as claimsDashboardLwc and check the isExposed checkbox.
- Under target configurations, select Lightning App Page, Home Page and Record Page.
- Click on Deploy.
- Open the HTML, JS and meta-xml files and paste the following code:

claimsDashboardLwc.html

```
<template>
  <lightning-card title="My Assigned Claims Dashboard" icon-name="standard:claim">
    <div class="slds-m-around_medium">
      <lightning-combobox
        name="policyFilter"
        label="Filter by Policy Type"
        value="All"
        options={policyTypeOptions} onchange={handleFilterChange}
        class="slds-m-bottom_medium">

      </lightning-combobox>
      <template if:true={visibleClaims}>
        <div class="slds-grid slds-wrap slds-grid_pull-padded">
          <template for:each={visibleClaims} for:item="claim">
            <div key={claim.claimId} class="slds-col slds-size_1-of-1 slds-medium-size_1-of-2
            slds-large-size_1-of-3 slds-p-horizontal_small">
              <c-claim-tile-lwc claim-data={claim}></c-claim-tile-lwc>

            </div>
          </template>
```

```
            </div>
            <p class="slds-m-top_medium slds-text-heading_small">Total Claims Assigned:
            {visibleClaims.length}</p>
          </template>
          <template if:false={visibleClaims}>
            <p>No claims currently assigned to you.</p>

          </template>
        </div>

    </lightning-card>

</template>
```

claimsDashboardLwc.js

```
import { LightningElement, wire, track } from 'lwc';
// Import the Apex method import getAssignedClaims from
'@salesforce/apex/ClaimsAdjusterController.getAssignedClaims';

export default class ClaimsDashboardLwc extends LightningElement {
    // List to hold all data retrieved from Apex
    @track allClaims = [];
    // List to display after
    filtering @track
    visibleClaims = []; error;

    // Wire service to call the Apex method
    @wire(getAssignedClaims)
    wiredClaims({ error, data }) {
        if (data) {
            this.allClaims = data;
            this.visibleClaims = data; // Initially display all claims
            this.error = undefined;
        } else if (error) {
            this.error = error;
            this.allClaims =
            undefined;
            this.visibleClaims =
            undefined;
            console.error('Error retrieving claims:', JSON.stringify(error));
        }
    }

    // Client-side filtering logic
    handleFilterChange(event) {
        const selectedPolicyType =
        event.target.value; if (selectedPolicyType
        === 'All') {
            this.visibleClaims = this.allClaims;

        } else {
```

```
                // Filter claims where policyType matches the selected value
                this.visibleClaims = this.allClaims.filter(claim =>
                    claim.policyType === selectedPolicyType
                );
            }
        }

        get policyTypeOptions() {
            // Options for the filter dropdown return [
                { label: 'All Policy Types', value: 'All' },
                { label: 'Auto', value: 'Auto' },
                { label: 'Property', value: 'Property' },
                { label: 'Life', value: 'Life' }
            ];
        }
}
```

claimsDashboardLwc.js-meta.xml

```
<?xml version="1.0"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>63.0</apiVersion>
    <isExposed>true</isExposed>
    <targets>
        <target>lightning__AppPage</target>
        <target>lightning__HomePage</target>
    </targets>
</LightningComponentBundle>
```

3. Create a reusable component (claimTileLwc) to display essential information for each claim

- Open the Lightning Studio extension from your Salesforce org.
- Click on Create New and select Lightning Web Component.
- Enter the component name as claimTileLwc and check the isExposed checkbox.
- Under target configurations, select Lightning App Page, Home Page and Record Page.
- Click on Deploy.
- Open the HTML, JS and meta-xml files and paste the following code:

claimTileLwc.html

```
<template>
    <lightning-layout-item size="12" padding="around-small">
        <div class="tile slds-box slds-p-around_medium slds-theme_default">
            <header class="slds-media slds-media_center slds-m-bottom_small">
                <div class="slds-media__figure">
```

```
                <lightning-icon icon-name={policyIcon} alternative-text={claimData.policyType}
                size="small"></lightning-icon>
            </div>
            <div class="slds-media__body slds-m-left_small">
                <h2 class="slds-text-heading_medium slds-m-bottom_none">
                    Claim: {claimData.claimNumber}
                </h2>
                <p class="slds-text-body_small">Policy Type:
                <strong>{claimData.policyType}</strong></p>
            </div>
        </header>
        <div class="slds-border_top slds-p-top_small">
            <p>Policy Holder: <strong>{claimData.policyHolderName}</strong></p> <lightning-
            formatted-number
                value={claimData.claimAmount}
                format-style="currency" currency-
                code="USD" label="Claim
                Amount">
            </lightning-formatted-number>
            <p>Status: {claimData.status}</p>
            <p class="slds-text-body_regular">
                Days Open: <strong class="slds-text-color_error">{claimData.daysOpen}</strong>
            </p>
        </div>
    </div>
</lightning-layout-item>
</template>
```

claimTileLwc.js

```
import { LightningElement, api } from 'lwc';

export default class ClaimTileLwc extends LightningElement {
    // @api decorator exposes this property to the parent component @api claimData;

    // Helper property to determine the correct icon based on Policy Type get policyIcon() {
        if (this.claimData && this.claimData.policyType) {
            erCase();
            'property') return
            ected_apps';
        }
        return 'utility:help';
    }
}
```

claimTileLwc.js-meta.xml

```
<?xml version="1.0"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>63.0</apiVersion>
    <isExposed>true</isExposed>
</LightningComponentBundle>
```

[Screenshot to be added: Claims Adjuster Dashboard (claimsDashboardLwc) output]

Milestone 4: Advanced Claim Processing, Security & Code Setup

1. Modify the Claim object to include an Approval Status field

- Go to Setup, click on Object Manager, search for the Claim object and click on it.
- In the left side panel, click on Fields and Relationships and click on New.
- Select Data Type as Picklist and label it as Approval Status.
- Select the Enter values radio button and enter New, Submitted for Approval, Approved and Rejected line by line.
- Field Name will be auto populated; click on Next, Next and Save.

2. Configure the Approval Process (Senior Adjuster → Department Manager, two sequential steps)

- Go to Setup, search for Approval Process in the Quick Find box.
- Select the Claim object for Manage Approval Processes For.
- Click on Create New Approval Process and select Use Standard Setup Wizard.
- Give the process name as High Value Claim Approval (Unique Name auto populated) and click on Next.
- Under Specify Entry Criteria, select Criteria Are Met with: Claim Amount greater than 50000. Click on Next.

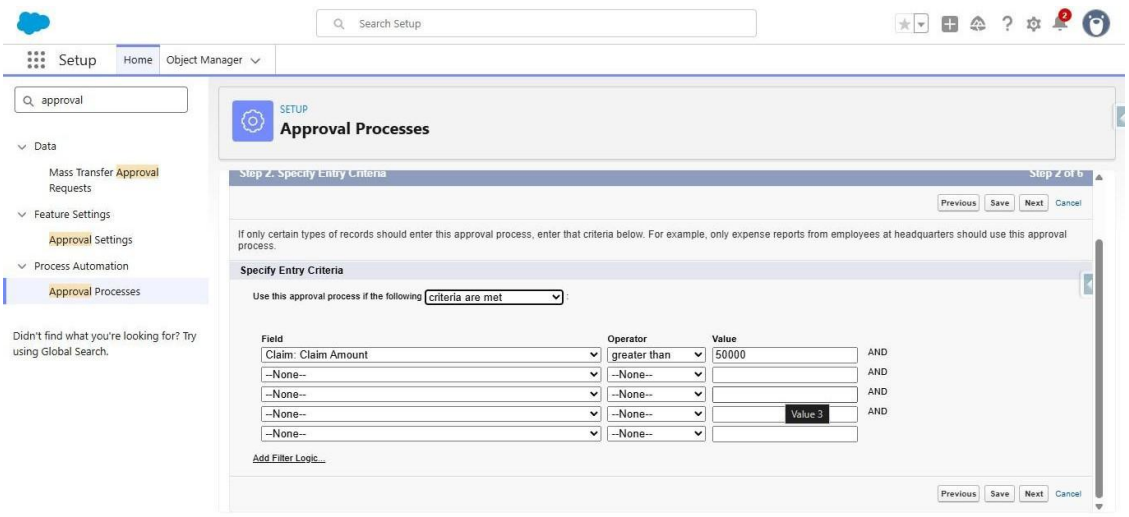


Figure 15: High Value Claim Approval - entry criteria (Claim Amount > 50000)

- Under Record Editability Properties, select Administrators ONLY can edit the records during the approval process. Click on Next twice.
- Under Approval Page Layout, move Claim Number, Owner, Claim Amount, Adjuster and Policy to the selected fields.
- Under Security Settings, select Allow Approvers to access the page only from within the Salesforce application. Click on Next.
- Under Initial Submitters, search for Owner under submitter type and select the claim owner as an allowed submitter. Click on Save.

2.1 Create Approval Steps

Step 1:

- Under Approval Steps, click on New and name it Step1 Senior Adjuster (Unique Name auto populated). Click on Next.
- Under Specify Step Criteria, select All records should enter this step. Click on Next.
- Under Select Approver, select Automatically assign to approver and choose the Senior Adjuster user.
- Under When multiple approvers are selected, select Approve or Reject based on the FIRST response.
- Click on Save.

Step 2:

- Click on New again and name it Step2 Manager Review. Click on Next.
- Under Specify Step Criteria, select All records should enter this step. Click on Next.
- Under Select Approver, select Automatically assign to approver and choose the Department Manager user.
- Under When multiple approvers are selected, select Approve or Reject based on the FIRST response.
- Under Reject Behavior, select Perform all rejection actions for this step AND all final rejection actions.

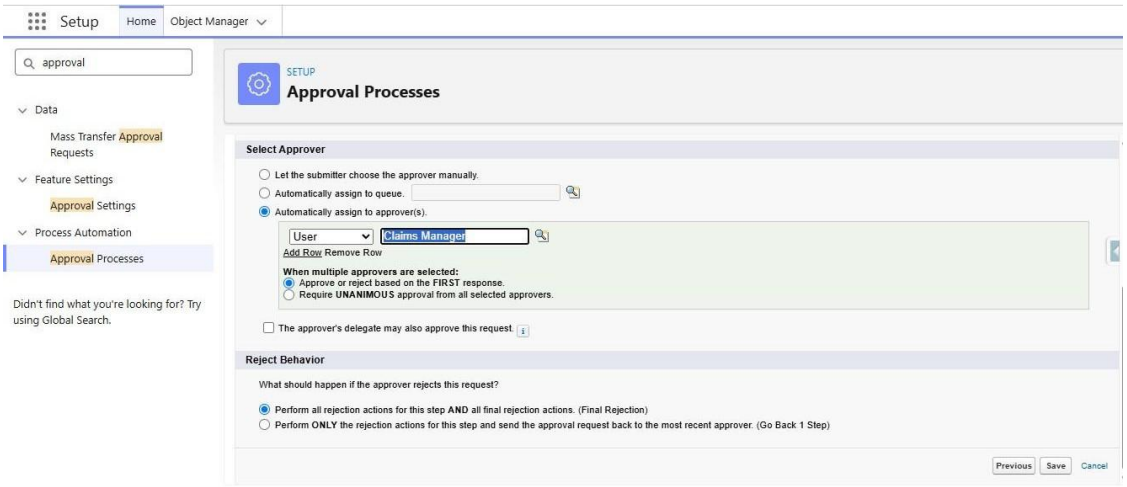


Figure 16: Step 2 Manager Review - approver and reject behavior

- Click on Save.

Field Updates:

- Initial Submission Actions → Add New → Field Update: name Update Status Submitted; object Claim; field Approval Status; Picklist; specific value Submitted for Approval; Save.
- Final Approval Actions → Add New → Field Update: name Update Status Approved; object Claim; field Approval Status; specific value Approved; Save.
- Final Rejection Actions → Add New → Field Update: name Update Status Rejected; object Claim; field Approval Status; specific value Rejected; Save.
- Activate the Approval Process.

3. Create a Record-Triggered Flow (After Save) that submits claims above $50,000 to the Approval Process

- Go to Setup, search for Flows in the Quick Find box, click on New Flow and select Record-Triggered Flow.
- Select the Claim object and configure the trigger as: A record is created or updated.
- Select All Conditions Are True with: Claim Amount Greater Than 50000 AND Approval Status Equals New.
- For When to Run the Flow for Updated Records, select Every time a record is updated and meets the condition requirements.
- Optimize the flow for Actions and Related Records.

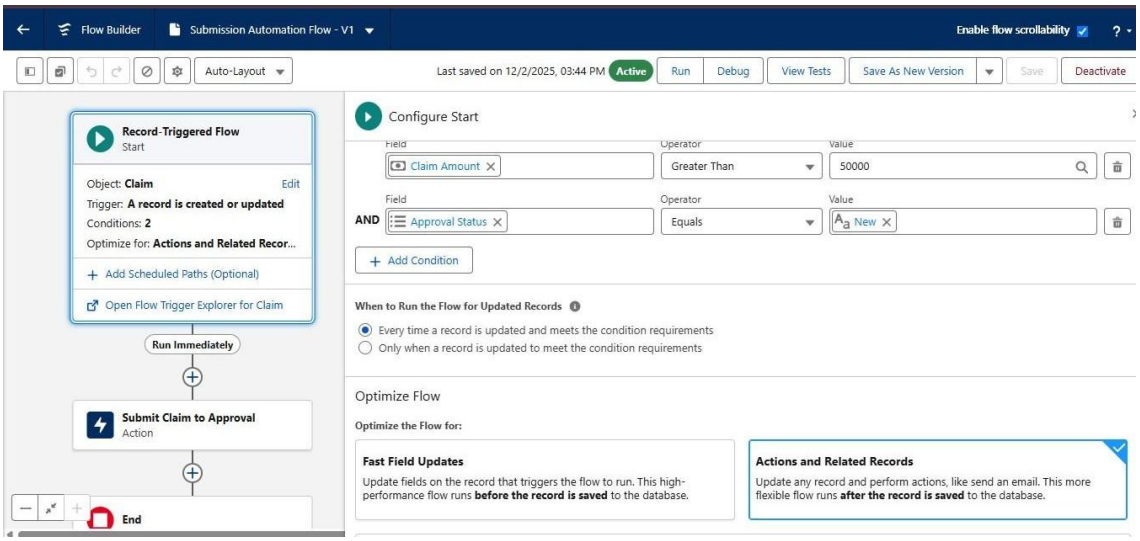


Figure 17: Submission Automation Flow - entry conditions

- Add an Action element after the Start element:
  - Label: Submit Claim to Approval.
  - Set Input Values: Record Id = Triggering Claim → Record ID. ◦

  Approval Process Name or Id = High Value Claim Approval.
- Save the flow as Submission Automation Flow and activate it.

4. Implement a Screen Flow for approvers to review and approve/reject from the Claim record

- Go to Setup, search for Flows, click on New Flow and choose Screen Flow.
- Create a new resource: Variable, API Name recordId, Data Type Text, and check Available for Input. Click on Done.
- Add a Get Records element: Label Get Claim Details; object Claim; condition Record ID equals variable recordId; store only the first record and automatically store all fields. • Add a Screen element labelled Claim Review Screen:
  - Add Display Text components showing: Claim ID {!Get_Claim_Details.Name}, Amount {!Get_Claim_Details.Claim_Amount__c}, Policy Type {!Get_Claim_Details.Policy__r.RecordType.Name}.

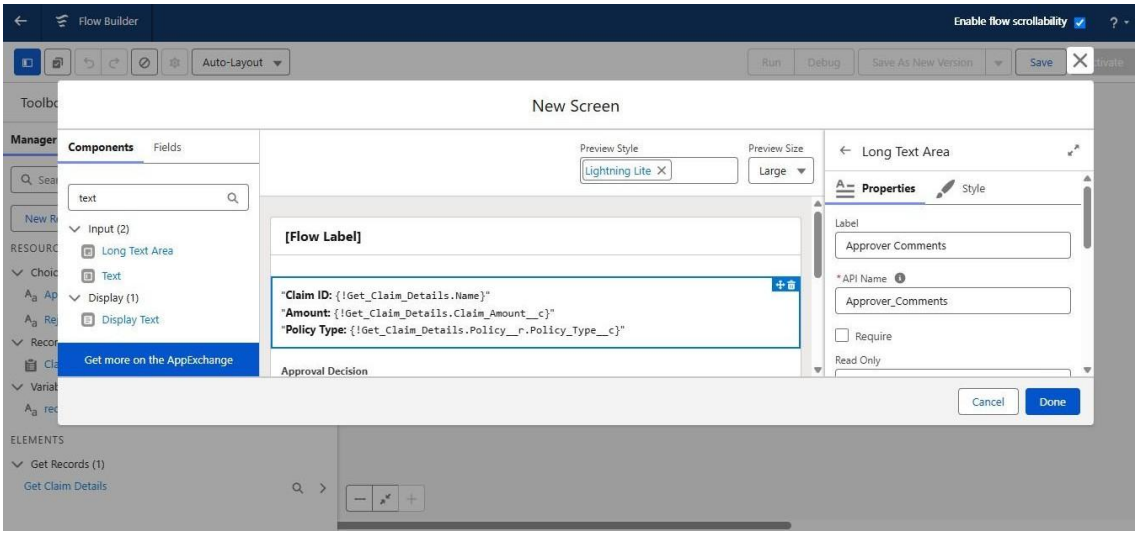


Figure 18: Claim Review Screen in the approver Screen Flow

  - Add a Radio Buttons component labelled Approval Decision, with two choice resources (Text): Approve and Reject.
  - Add a Long Text Area component labelled Approver Comments. Click on Done.
- Add a Decision element labelled Check Decision:
  - Outcome 1: Decision Approved – Approval Decision from Claim Review Screen equals Approve.
  - Default Outcome: Decision Rejected.
- Under the Decision Approved path, add the Submit for Approval action: label Approve Claim; Record ID = recordId variable; submission comments = Approver Comments from Claim Review Screen.
- Under the Decision Rejected path, add the Submit for Approval action: label Reject Claim; Record ID = recordId variable; submission comments = Approver Comments from Claim Review Screen.
- Save and activate the flow with the name Claim Approver Screen Flow.

4.1 Create a Quick Action

- Go to Setup, click on Object Manager, search for the Claim object and select it.
- In the left side panel, click on Buttons, Links and Actions and click on New Action.
- Select Action Type as Flow and select the flow you have just created.
- Label it as Approve/Reject Claim (Name auto populated) and click on Save.

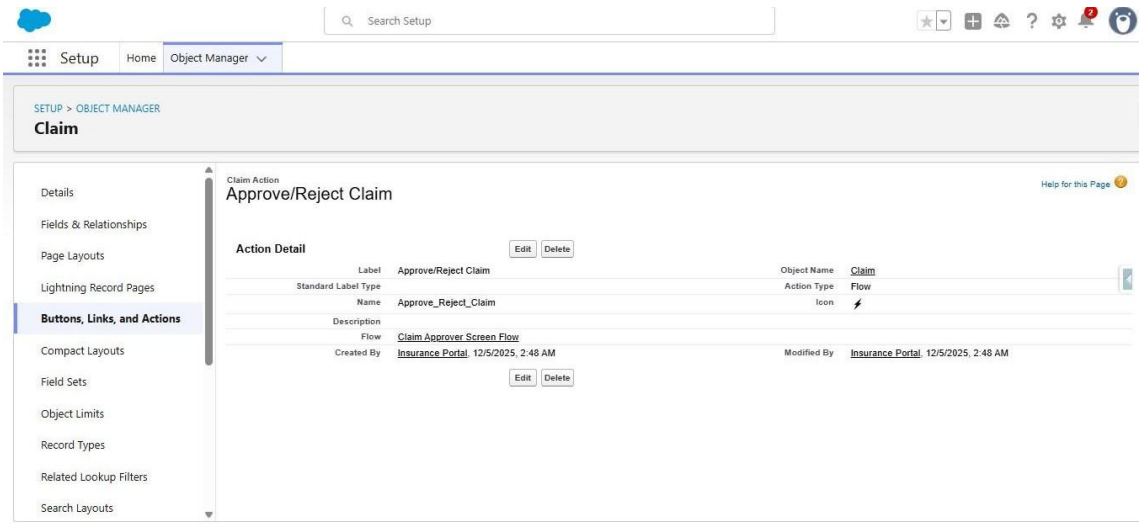


Figure 19: Approve/Reject Claim quick action

- Add the quick action to the Claim page layout: open Page Layouts → Claim page layout → Mobile & Lightning Actions, then drag Approve/Reject Claim to the Salesforce Mobile and Lightning Experience Actions section and click on Save.

5. Write an extensive Apex Test Class for the ClaimsAdjusterController (95%+ code coverage)

- Click on the Setup icon, right-click on Developer Console and open the link in a new tab.
- Click on File → New → Apex Class and name it ClaimsAdjusterControllerTest.
- Paste the following code:

ClaimsAdjusterControllerTest.cls

```
@isTest
private class ClaimsAdjusterControllerTest {
    @testSetup
    static void setupTestData() {
        // Create a real active test user
        Profile p = [SELECT Id FROM Profile WHERE Name = 'Standard User' LIMIT 1]; User
        u = new User(
            FirstName = 'Test',
            LastName = 'User',
            Alias = 'tuser',
            Username = 'testuser1234343@example.com.test',
            Email = 'testuser123@example.com',
            TimeZoneSidKey = 'America/Los_Angeles',
            LocaleSidKey = 'en_US',
```

```
            EmailEncodingKey = 'UTF-8',
            LanguageLocaleKey = 'en_US',
            ProfileId = p.Id
        );
        insert u;

        // Query Policy__c and Claim__c RecordTypes
        Id policyRT = Schema.SObjectType.Policy__c.getRecordTypeInfosByDeveloperName()
            .get('Auto').getRecordTypeId();
        Id claimRT = Schema.SObjectType.Claim__c.getRecordTypeInfosByDeveloperName()
            .get('Accident').getRecordTypeId();

        // Create Contact owned by same user
        Contact con = new Contact(FirstName = 'John', LastName = 'Doe', OwnerId = u.Id);
        insert con;

        // Create Policy owned by same user
        Policy__c policy = new Policy__c(
            Customer__c = con.Id,
            RecordTypeId = policyRT,
            OwnerId = u.Id,
            VIN__c = '1A903284375893532'
        );
        insert policy;

        // Create Claim owned by same user
        Claim__c claim = new Claim__c(
            Policy__c = policy.Id,
            RecordTypeId = claimRT,
            Claim_Amount__c = 5000,
            Date_of_Loss__c = Date.today().addDays(-10),
            OwnerId = u.Id
        );
        insert claim;
        System.debug('Claim inserted: ' + claim.Id);
    }

    @isTest
    static void testGetAssignedClaims() {
        User u = [SELECT Id FROM User WHERE Alias = 'tuser' LIMIT 1];
        System.runAs(u) {
            Test.startTest();
            List<ClaimsAdjusterController.ClaimWrapper> results =
                ClaimsAdjusterController.getAssignedClaims();
            System.debug('Result is: ' + results);
            Test.stopTest();

            ClaimsAdjusterController.ClaimWrapper wrap = results[0];
            System.assertNotEquals(null, wrap.claimId);
            System.assertNotEquals(null, wrap.claimNumber);
```

```
        System.assertEquals('New', wrap.status);
        System.assertEquals(5000, wrap.claimAmount);
        System.assert(wrap.policyHolderName.contains('John'));
        System.assertNotEquals('Unknown',  wrap.policyType);
    }

  }

}
```

- Save the file using Ctrl + S.
- On the top right corner, click on Run Test to run the test class.
- To check the test results, open the Tests tab at the bottom and view the percentage of the Apex class covered.

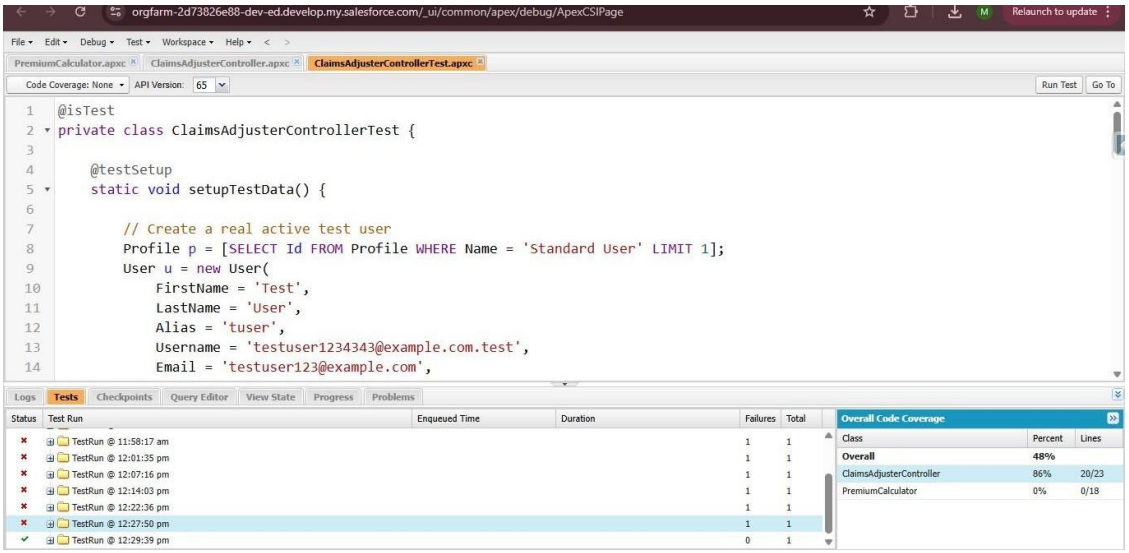


Figure 20: ClaimsAdjusterControllerTest run in Developer Console

6. Refine the Sharing Rules so adjusters only see claims in their assigned territory/state

A Record-Triggered Flow first copies the policy state onto each Claim, so that sharing rules can use it as criteria.

- Go to Setup, search for Flows, click on New Flow and select Record-Triggered Flow.
- Select the Claim object and configure the trigger as: A record is created or updated; entry conditions: none; optimize the flow for Updates.
- Add a Get Records element: Label get_policy_records; filter None; store all the records; choose fields and let Salesforce do the rest; select the Policy State field.
- Add a Loop element after Get Records: Label Claim_Filed_update; collection variable = policies from Get Policy Records; direction: first item to last item.
- Under For Each, add an Update Records element: Label Claim_Filed_state_update; filter conditions None; set the Policy Account Holder State field on the claim from the Policy State of the current loop item.
- Save and activate the flow with the name Claim Policy Holder State Update.

6.1 Define Sharing Rules for Claim object

- Go to Setup, search for Sharing Settings in the Quick Find box and click on it.
- Under Manage Sharing Settings For, select the Claim object and scroll down to Claim Sharing Rules.
- Click on New: Label Share Records based on Policy State CA; criteria Policy Account Holder State Equals CA; Access Level Read/Write; Save.

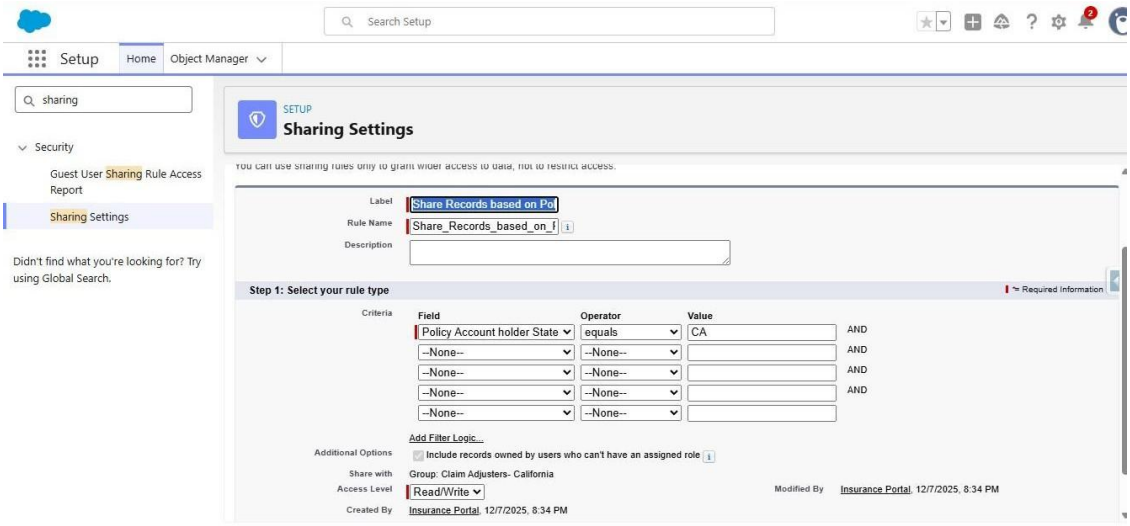


Figure 21: Sharing rule for Policy State CA

- Click on New again: Label Share Records based on Policy State TX; criteria Policy Account Holder State Equals TX; Access Level Read/Write; Save.

7. Finalize Permission Sets to differentiate access (Agents, Adjusters, Managers)

- Go to Setup, search for Permission Sets in the Quick Find box and click on it. Click on New.
- Label: Insurance Agent Access (API Name auto populated).
- Object access: Create and Read for Policy; Read and Edit for Contact; Read only for Claim.
- System permissions: make Flow User enabled.
- Apex Class access: enable PremiumCalculator.

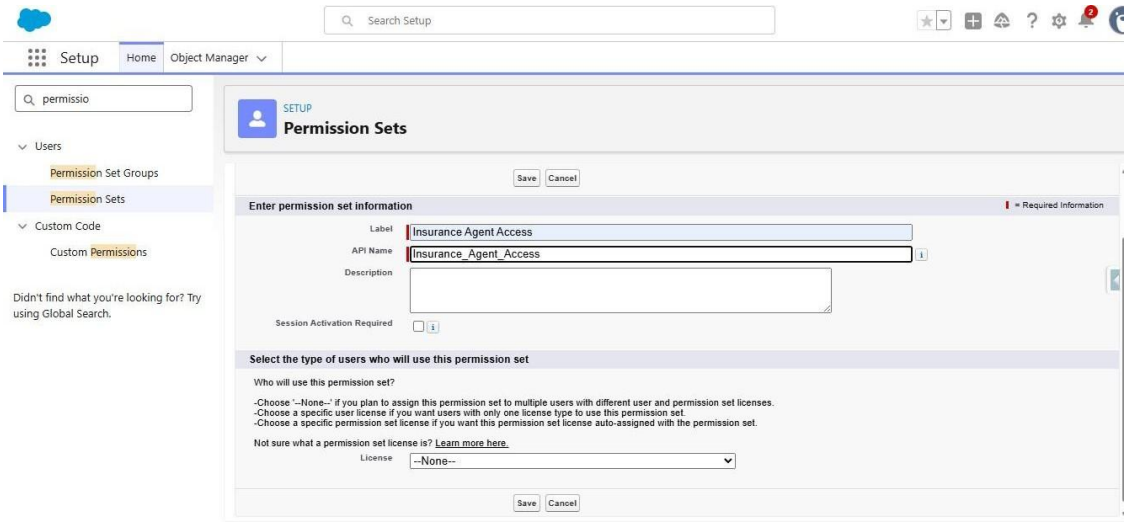


Figure 22: Insurance Agent Access permission set

7.1 Create Claims Manager Access Permission Set

- Click on New and label it as Claims Manager Access (API Name auto populated).
- Object access: Read only for Policy; Read and Edit for Contact; Read, Edit and Delete for Claim.
- System permissions: enable Run Reports and Manage Approvals.

7.2 Create Claims Adjuster Access Permission Set

- Click on New and label it as Claims Adjuster Access (API Name auto populated).
- Object access: Read only for Policy; Read and Edit for Claim.
- Apex Class access: enable ClaimsAdjusterController.

7. FUNCTIONAL & PERFORMANCE TESTING

During testing, the following features were verified in the Salesforce org to ensure correct functionality and performance. Screenshots of each test result should be attached below the corresponding test case.

Test ID	Feature Tested	Test Description	Expected Result
TC-01	Policy & Claim objects	Create a Policy and a Claim record with auto-number names.	Records are created as P-0001 and C-0001 format.
TC-02	Record Types & Field Sets	Create Auto, Property and Life policies and view product-specific fields.	Correct record type and fields are displayed.
TC-03	AutoQuoting Screen Flow	Run the flow with policy holder, start date, state, VIN and model year.	Draft Policy record is created.
TC-04	VIN Validation Rule	Save an Auto policy with a VIN not equal to 17 characters.	Error message is displayed on the VIN field.

Test ID	Feature Tested	Test Description	Expected Result
TC-05	Premium Calculation	Run the flow for CA/TX policies and model years before 2018.	Premium is calculated and updated on the policy.
TC-06	Claim Routing Flow	Create claims against Auto, Property and Life policies.	Claim owner is set to the matching queue.
TC-07	High-Value Approval	Create a claim with an amount above 50,000.	Claim is submitted to Senior Adjuster, then Manager.
TC-08	Approve/Reject Screen Flow	Use the quick action to approve and reject a claim.	Approval Status is updated with comments.
TC-09	Claims Dashboard (LWC)	Open the dashboard and filter by All, Auto, Property, Life.	Claim tiles display and filter correctly.
TC-10	Apex Test Class	Run ClaimsAdjusterControllerTest.	Tests pass with required code coverage.
TC-11	Sharing Rules & Permission Sets	Log in as Agent, Adjuster and Manager users.	Each user sees only the permitted objects and records.

8. ADVANTAGES & DISADVANTAGES

Advantages

- **Centralized Platform** – Manages Vehicle, Property, and Life Insurance in one Salesforce system.
- **Automated Quoting** – Screen Flows simplify and standardize policy quoting.
- **Automated Premium Calculation** – Apex calculates premiums consistently.
- **Automated Claim Routing** – Claims are routed to the appropriate queue based on policy type.
- **Faster Claim Processing** – Approval and escalation processes reduce manual effort.
- **Better Visibility** – LWC Claims Dashboard gives adjusters a focused view of assigned claims.
- **Improved Data Accuracy** – Validation rules help maintain accurate policy information.
- **Flexible Data Model** – Record Types and Field Sets support different insurance products.
- **Secure Access** – Permission Sets and Sharing Rules support role- and state-based access.
- **Scalable Solution** – The architecture can support multiple insurance lines and future enhancements.

Disadvantages

- **Complex Data Management** – Vehicle, Property, and Life policies require different data fields.
- **Approval Complexity** – Claim approval must consider claim amount, policy type, and adjuster level.
- **User Adoption** – Agents need intuitive flows to minimize clicks and improve efficiency.
- **Integration Complexity** – External database callout simulation requires additional Apex and mock test classes.
- **Salesforce Dependency** – The solution depends on Salesforce configuration and platform capabilities.
- **Implementation Effort** – Multiple Flows, Apex classes, LWC components, security rules, and approval processes require careful development and maintenance.

9. CONCLUSION

The Multi-Line Insurance Policy and Claims Management System provides a centralized Salesforce-based solution for managing Vehicle, Property, and Life Insurance policies and claims. The project addresses the limitations of manual and fragmented processes by introducing guided quoting, automated premium calculation, claim routing, structured approval processes, and an LWC-based Claims Dashboard.

The solution uses a combination of Salesforce Flows, Apex, Record Types, Field Sets, Validation Rules, Approval Processes, LWC, Permission Sets, and Sharing Rules to improve policy processing, claim handling, data accuracy, security, and operational visibility.

Overall, the project demonstrates how Salesforce can bring multiple insurance processes together on a single scalable platform, helping insurance teams work more efficiently while providing better visibility and service to customers.

10. FUTURE SCOPE

The system can be further enhanced in the following areas:

- **External System Integration** – Integrate with external vehicle and property databases for real-time quote verification.
- **Additional Insurance Products** – Extend the solution to support additional insurance lines beyond Vehicle, Property, and Life.
- **Advanced Claims Automation** – Introduce more automated claim assessment and escalation based on claim characteristics.
- **Enhanced Analytics** – Provide advanced dashboards and reports for policy, premium, and claims analysis.
- **Improved Customer Experience** – Introduce customer-facing capabilities for policy and claim status tracking.

- **Mobile Accessibility** – Enable agents and claims users to access key Salesforce processes from mobile devices.
- **AI-Based Assistance** – Explore AI capabilities to assist agents and adjusters with policy and claims-related tasks.
- **Advanced Integration Architecture** – Expand API-based integrations with external insurance and verification systems.

11. APPENDIX

- **Objects** – Policy, Claim, Contact, User
- **Record Types** – Vehicle (Auto), Property, Life; Accident, Property, Life
- **Automation** – Screen Flows, Record-Triggered Flows, Approval Processes
- **Apex** – PremiumCalculator, ClaimsAdjusterController
- **LWC** – Claims Dashboard (claimsDashboardLwc), Claim Tile (claimTileLwc)
- **Validation** – 17-character VIN validation
- **Security** – Permission Sets, State-Based Sharing Rules
- **Testing** – ClaimsAdjusterControllerTest
- **Reporting** – Claims Dashboard & Salesforce Reports/Dashboards